

Modélisation Avancée

Matisse

kra

bts sio

Sommaire

Associations Ternaires	2
Objectif du TD :	2
1. Phase de Modélisation (MCD et MLD)	2
3. Requêtes SQL	3
Conclusion	5
Les associations réflexives	6
1. Contexte du projet	6
2. Modélisation Conceptuelle et Logique (Étapes 1 à 3)	6
3. Création et Alimentation de la Base de Données	6
4. Requêtage et Extraction de données	7
Conclusion et Bilan du TP	8
L'héritage	9
1.Objectif du TD :	9
2.création de l'héritage	9
conclusion	10
relation multiple	11
1.Objectifs du TP	11
2.Modélisation Conceptuelle (MCD)	11
3.base de donnée	12
Conclusion	13

Associations Ternaires

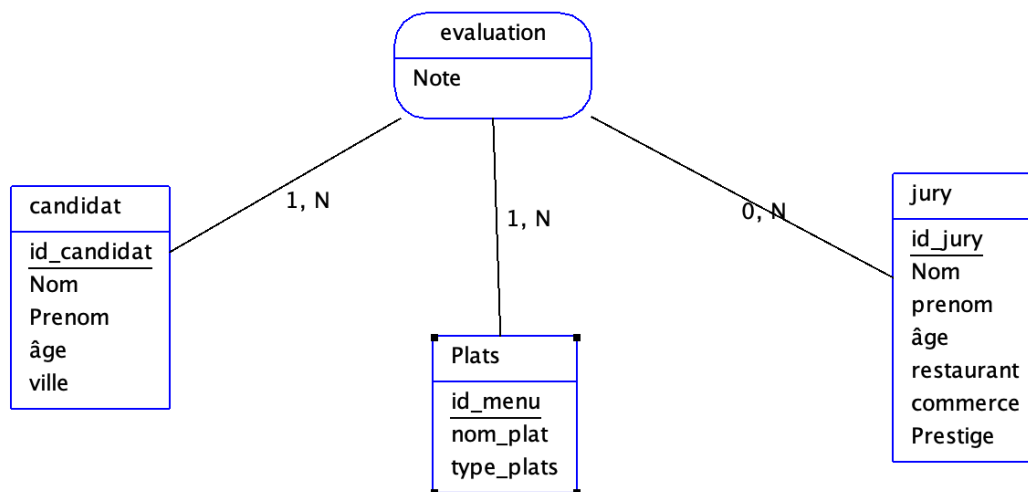
Objectif du TD :

Comprendre, modéliser et implémenter une base de données relationnelle à partir d'un cahier des charges impliquant des associations de dimension 3 (ternaires), puis interroger cette base avec des requêtes SQL complexes.

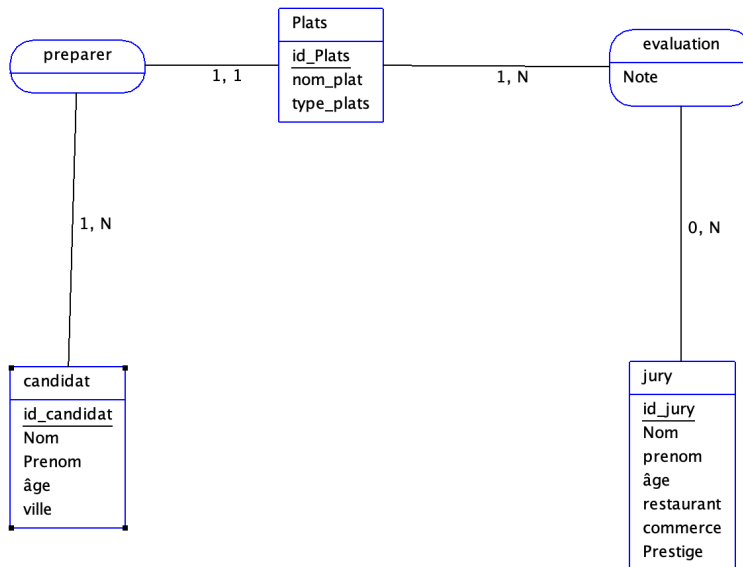
1. Phase de Modélisation (MCD et MLD)

La première étape s'est concentrée sur la conception de la base de données pour l'émission de cuisine « ChefauTop ».

- Gestion de l'association ternaire : Nous avons identifié que l'action d'évaluer un plat reliait trois entités distinctes : le Candidat, le Jury et le Plat et La propriété Note a été correctement placée au centre de cette association ternaire



- Simplification du Modèle Conceptuel de Données (MCD) : Nous avons ensuite restructuré le MCD en supprimant l'association ternaire au profit d'associations binaires. L'entité Plat est devenue le point central, reliée d'un côté au créateur (Candidat via 1,1 - 1,N) et de l'autre à l'évaluateur (Jury via 1,N - 0,N portant la note).



- Passage au Modèle Logique de Données (MLD) : Les règles de passage du modèle conceptuel au modèle logique ont été appliquées pour définir les clés étrangères et créer une table de liaison NOTER composée des clés primaires des entités reliées.

ancien mld

```

# Modèle créé le : Mon Mar 16 13:41:30 CET 2026
candidat (id_candidat_candidat, Nom_candidat, Prenom_candidat, âge_candidat, ville_candidat)
jury (id_jury_jury, Nom_jury, prenom_jury, âge_jury, restaurant_jury, commerce_jury, Prestige_jury)
Plats (id_menu_menu, nom_plat_Plats, type_plats_Plats)
evaluation (id_candidat_candidat, id_jury_jury, id_menu_menu, Note_evaluation)
    
```

nouveau mld

```

# Modèle créé le : Mon Mar 16 13:53:01 CET 2026
candidat (id_candidat_candidat, Nom_candidat, Prenom_candidat, âge_candidat, ville_candidat)
jury (id_jury_jury, Nom_jury, prenom_jury, âge_jury, restaurant_jury, commerce_jury, Prestige_jury)
Plats (id_Plats, nom_plat_Plats, type_plats_Plats, #id_candidat_candidat)
evaluation (id_jury_jury, id_Plats, Note_evaluation)
    
```

2. Phase d'Implémentation Physique (SQL DDL & DML)

À partir du MLD, le script de création de la base de données a été rédigé pour le SGBD (MariaDB/MySQL via XAMPP).

- Création des tables (DDL) : Les tables indépendantes PLAT, CANDIDAT et JURY ont été créées en premier avec leurs clés primaires respectives. La table associative NOTER a été créée avec ses contraintes de clés étrangères (FOREIGN KEY).
- Insertion des données : Le jeu d'essai fourni dans l'énoncé a été inséré via des requêtes INSERT INTO afin de remplir la base pour les tests.

3. Requêtes SQL

La dernière partie du TD a permis de manipuler les données pour répondre à des problématiques précises. Plusieurs concepts SQL avancés ont été mis en pratique de manière combinée :

Fonctions d'agrégation : Utilisation de COUNT() pour dénombrer les notes totales.

```
SELECT COUNT(*) FROM NOTER;
```

☐ Profilage [Éditer en ligne] [Éditer]

Options supplémentaires

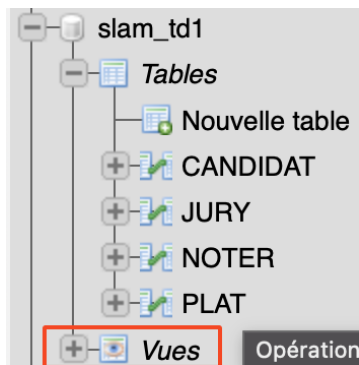
COUNT(*)

45

Création de Vues (CREATE VIEW) : Mise en place de tables virtuelles pour stocker des calculs intermédiaires réutilisables.

- Création d'une vue vue_moyennes pour calculer la moyenne (AVG) des notes groupées par candidat (GROUP BY IdCandidat).

```
1 CREATE VIEW vue_moyennes AS
2 SELECT IdCandidat, AVG(note) AS moyenne_notes
3 FROM NOTER
4 GROUP BY IdCandidat;
```



- Création d'une vue vue_moyennes_jury pour calculer la moyenne des notes données par chaque membre du jury.

```
1 CREATE VIEWS `vue_moyennes_jury` AS
2 SELECT id AVG(note) AS moyenne notes
3 FROM NOTER
4 GROUP by id;
```

idJury	moyenne_notes
1	13.0000
2	13.2222
3	8.3333
4	11.6667
5	17.4444

Jointures (JOIN) et Sous-requêtes : Élaboration de requêtes complexes croisant les tables physiques et les vues. L'objectif était d'extraire des informations textuelles (noms et prénoms) conditionnées par des requêtes de sélection :

- Identification du grand vainqueur en comparant les moyennes avec (SELECT MAX(moyenne_notes)).

```
1 SELECT CANDIDAT.nom, CANDIDAT.prenom, vue_moyennes.moyenne_notes
2 FROM vue_moyennes
3 JOIN CANDIDAT ON vue_moyennes.IdCandidat = CANDIDAT.id
4 WHERE vue_moyennes.moyenne_notes = (SELECT MAX(moyenne_notes) FROM vue_moyennes);
```

nom	prenom	moyenne_notes
touille	rata	13.0667

- Identification du membre du jury le plus sévère avec (SELECT MIN(moyenne_notes)).

```
1 SELECT JURY.nom, JURY.prenom, vue_moyennes_jury.moyenne_notes
2 FROM vue_moyennes_jury
3 JOIN JURY ON vue_moyennes_jury.idJury = JURY.id
4 WHERE vue_moyennes_jury.moyenne_notes = (SELECT MIN(moyenne_notes) FROM
vue_moyennes_jury);
```

nom	prenom	moyenne_notes
Ducasse	Alain	8.3333

- Identification du membre du jury le plus clément avec (SELECT MAX(moyenne_notes))

```
1 SELECT JURY.nom, JURY.prenom, vue_moyennes_jury.moyenne_notes
2 FROM vue_moyennes_jury
3 JOIN JURY ON vue_moyennes_jury.idJury = JURY.id
4 WHERE vue_moyennes_jury.moyenne_notes = (SELECT MAX(moyenne_notes) FROM vue_moyennes_jury);
```

nom	prenom	moyenne_notes
Tout	Bouffe	17.4444

Conclusion

Ce TD a permis de valider la transition d'une modélisation théorique sur papier vers une base de données physique concrète et fonctionnelle. La simplification des associations ternaires a rendu l'architecture de la base évolutive et robuste.

La maîtrise de ces jointures, vues et sous-requêtes constitue une architecture de données fiable, prête à être connectée et exploitée par des procédures algorithmiques ou une future application orientée objet développée en langage Java.

Les associations réflexives

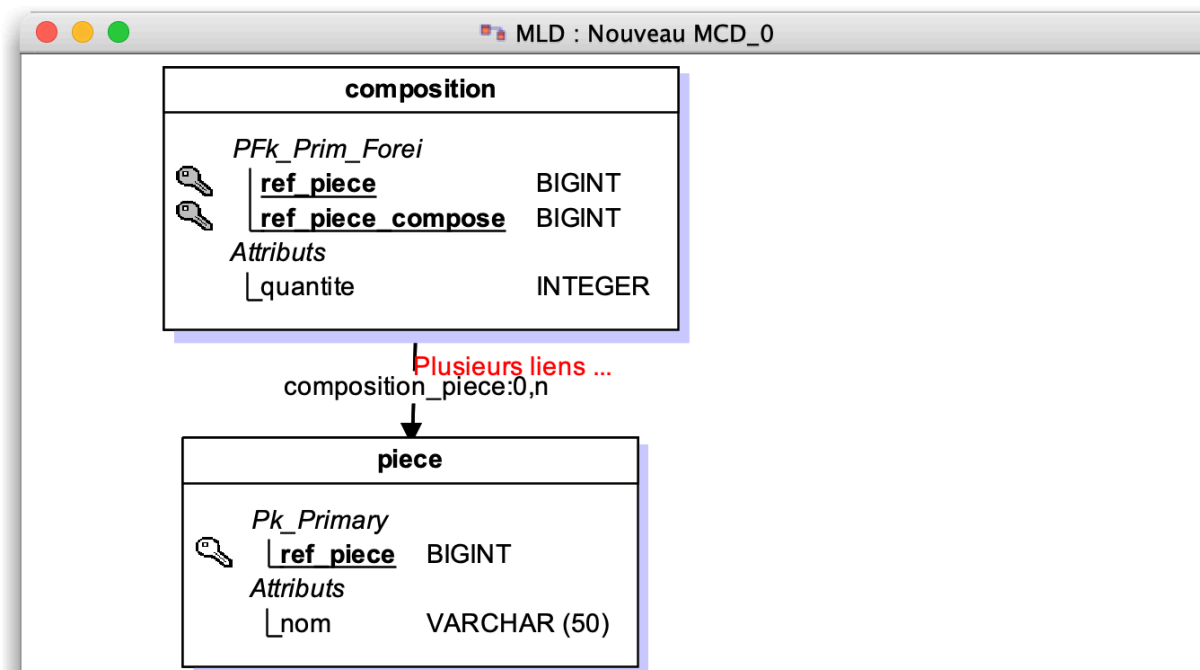
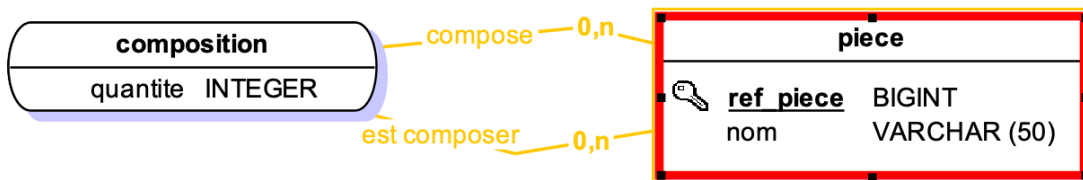
1. Contexte du projet

La société Aile3000 s'occupe de la réparation d'aile des avions. Afin d'optimiser cette activité, la gestion de ses pièces s'effectue dans une base de données. La règle de gestion principale est la suivante : une pièce peut entrer dans la composition de plusieurs pièces (sauf d'elle-même...) et une pièce peut être composée de plusieurs autres pièces. Il s'agit donc de modéliser une nomenclature (ou vue éclatée) avec la notion de quantité.

2. Modélisation Conceptuelle et Logique (Étapes 1 à 3)

Le Modèle Conceptuel de Données (MCD)

Pour répondre à la règle de gestion, nous avons mis en place une association réflexive.



Le Modèle Logique de Données (MLD)

La transformation de l'association réflexive de type (n,n) génère une table intermédiaire contenant les deux clés étrangères pointant vers l'entité PIECE.

3. Création et Alimentation de la Base de Données

Matisse kra
bts sio

Après la création de la base de données SLAM1_TD2 , j'ai pu generer le script sql directement apres la verification de mon mcd le script a été exécuté en s'assurant d'utiliser le moteur de stockage InnoDB pour garantir l'intégrité référentielle

4. Requêtage et Extraction de données

Étape 10 : Afficher le nom et la quantité des pièces à commander pour une aile

Consigne : On ne détaillera pas les sous pièces.

```
SELECT piece.nom, composition.quantite FROM composition JOIN piece ON composition.Composantid = piece.pieceID WHERE composition.composeId= 'aile';
```

nom	quantite
Aileron d'aile	1
Longeron d'aile	5
rivets à la pelle	100
Train de machin	1

Étape 11 : Afficher le nombre total de pièces à commander pour une aile

Consigne : Sans prendre en compte les sous pièces. Ici, nous utilisons la fonction d'agrégation SUM() pour additionner le volume physique de pièces et non COUNT() qui compterait seulement les types de pièces.

```
1 SELECT SUM(Quantite)
2 FROM COMPOSITION
3 WHERE ComposeId = 'aile';
```

SUM(Quantite)

107

Étape 12 : Afficher le nom et la quantité des pièces à commander pour un "Train de machin"

Consigne : On part du principe qu'on ne connaît pas l'id de la pièce. Pour répondre à cette contrainte, l'utilisation d'une sous-requête dans la clause WHERE s'avère pertinente afin de retrouver dynamiquement l'identifiant à partir du nom de la pièce.

```
1 SELECT piece.nom, composition.quantite
2 FROM composition
3 JOIN piece ON composition.composantId = piece.pieceID
4 WHERE composition.composeId = (
5     SELECT pieceID
6     FROM piece
7     WHERE nom = 'Train de machin'
8 );
```

nom	quantite
Charnière à tout faire	3
rivets à la pelle	8

Conclusion et Bilan du TP

Ce TD m'a permis d'aborder une notion fondamentale et complexe de la modélisation de bases de données : l'association réflexive.

Compétences acquises : L'extraction de données sur une table réflexive m'a permis de pratiquer les jointures complexes et les sous-requêtes, notamment pour retrouver dynamiquement des informations sans connaître l'identifiant de départ (comme exigé à l'étape 12).

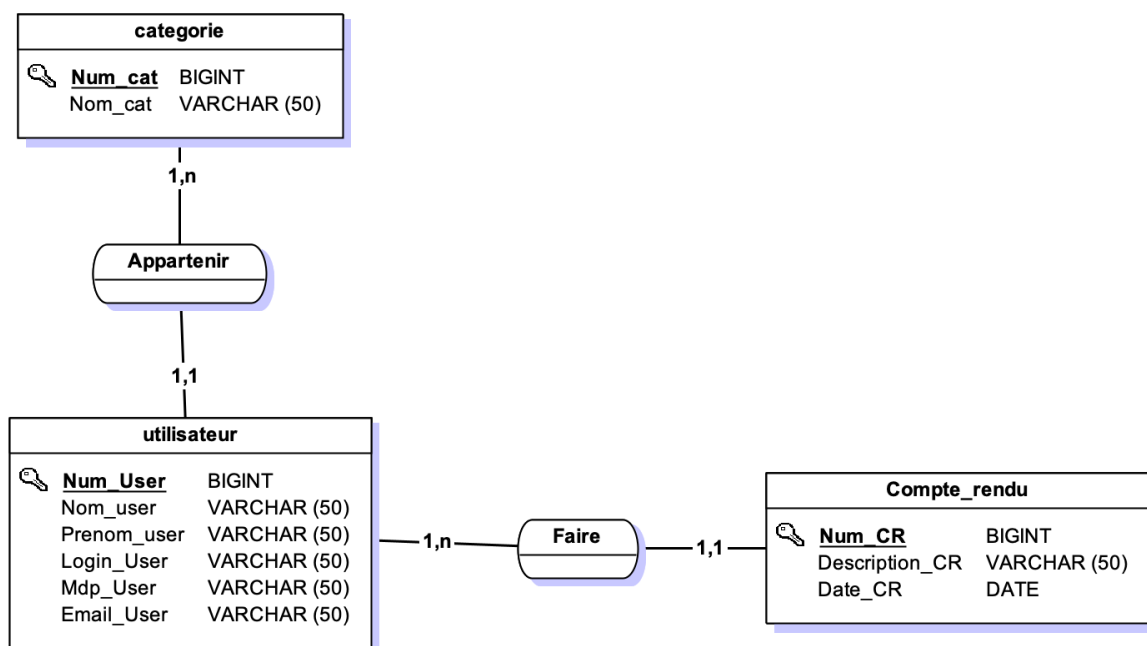
L'héritage

1.Objectif du TD :

l'objectif de ce tp est d'apprendre à manipuler l'héritage sur un mcd c'est à dire au lieu de faire plusieurs entités qui ont les mêmes attributs comment faire pour faire une seule entité qui va regrouper tous les attributs des autres entités

2.création de l'héritage

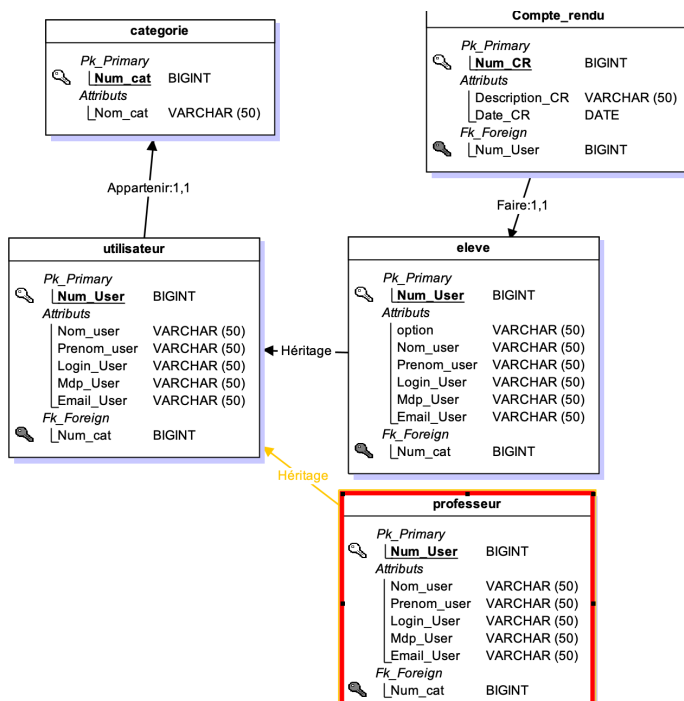
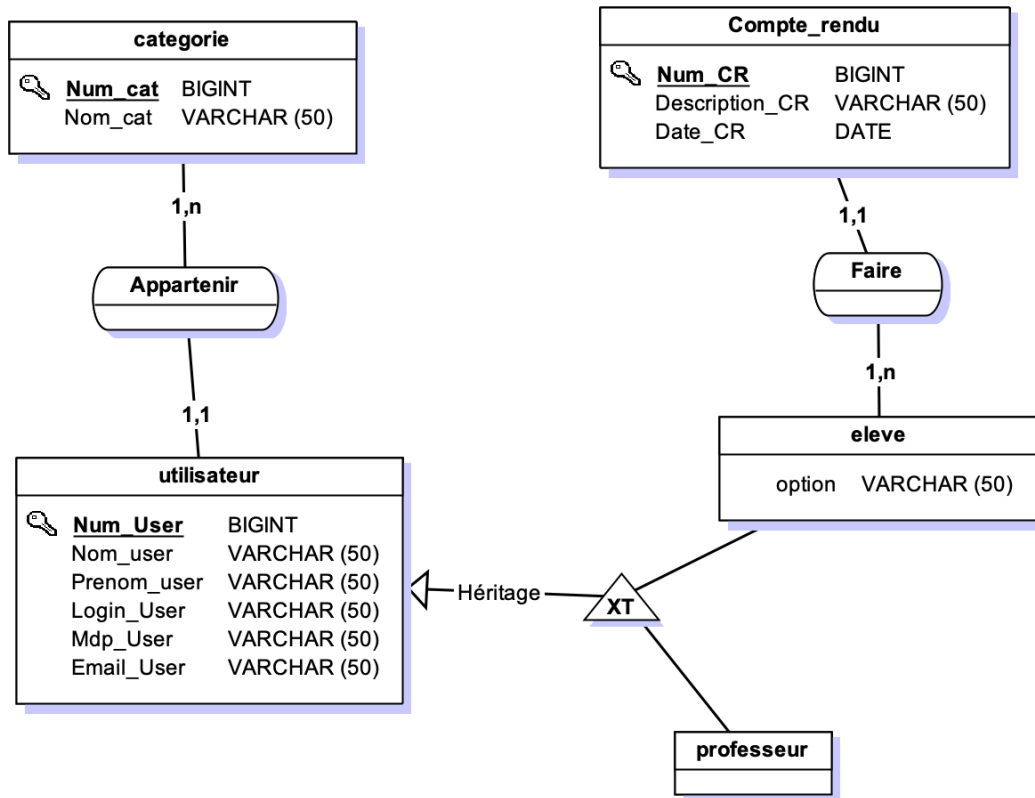
voici le mcd de base



Il nous est demandé de rajouter l'attribut option pour l'élève et que le compte rendu peut être fait seulement par un élève et non un professeur.

Pour répondre à ce besoin on va partir de mcd et transformer l'entité utilisateur en entité mère celle qui contient les attributs communs aux entités élève et professeur puis créer les entités filles entité élève et entité professeur. Dans l'entité élève on ajoute l'attribut option comme il nous est demandé puis on la relie à l'association faire et à l'entité utilisateur. L'entité professeur est seulement reliée à l'entité utilisateur.

Matisse kra
bts sio



conclusion

ce tp m'as permis d'acquérir des compétence approfondie à la réalisation de la modélisation d'un mcd en manipulant les entité mère et fille

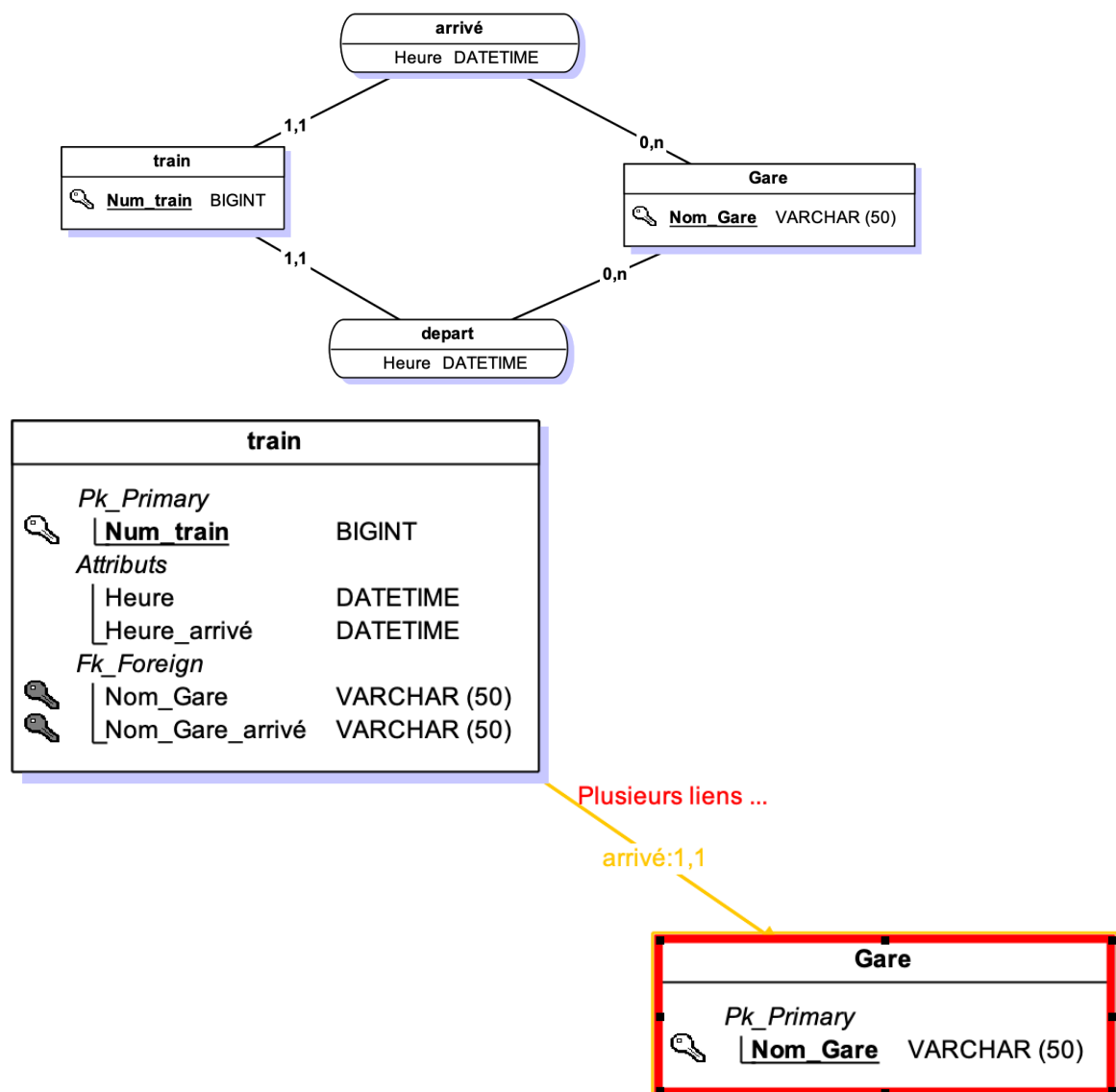
relation multiple

1.Objectifs du TP

L'objectif était de modéliser une base de données capable de gérer des trains, des lignes commerciales et des horaires de passage dans différentes gares. Le TP a évolué d'une relation simple (Départ/Arrivée) vers une gestion complète des gares desservies (étapes intermédiaires).

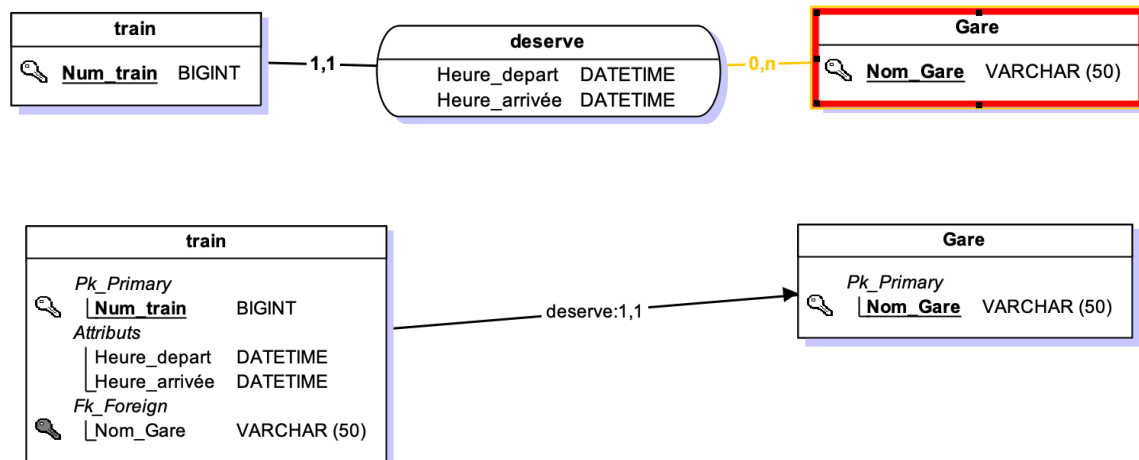
2.Modélisation Conceptuelle (MCD)

un premier modèle de mcd et mld a été créé avec deux association l'association arrivée et l'association départ



Matisse kra
bts sio

ensuite on s'est aperçu que la sncf souhaite enregistrer pour chaque train les gares desservies, ainsi que l'horaire d'arrivée de départ dans ces gares. nouveau mcd et mld ont été réalisés



3.base de donnée

dans la base de donnée j'ai implémenter deux tables de plus qui n'était pas demander dans la réalisation du mcd mais que j'ai jugé nécessaire pour répondre au problème final
j'ai créé les tables :

- ligne qui comprend la ligne empruntée par le train
- desservir les gare desservi par le train

j'ai ensuite implémenter dans ma base ces deux train

étape 7 : Insérer dans votre base de données le train n°242:

- ligne n°2 - vert : Simplon-Orient-Express
- départ de Paris le 10/03/2020 à 8h30
- arrivée à Istanbul le 15/03/2020 à 16h45
- tarif : 4000€
- gares desservies :
 - Lausanne : le 10/03/2020 à 16h15
 - Milan : le 11/03/2020 à 9h45
 - Venise : le 12/03/2020 à 8h
 - Belgrade : le 13/03/2020 à 19h55
 - Sofia : le 14/03/2020 à 16h

étape 8 : Insérer dans votre base de données le train n°422:

- ligne n°4 - violet : Venise-Simplon-Orient-Express
- départ de Calais le 10/03/2020 à 9h00
- arrivée à Venise le 12/03/2020 à 20h30
- tarif : 2360 €
- gares desservies :
 - Paris : le 10/03/2020 à 12h
 - Lausanne : le 11/03/2020 à 8h
 - Milan : le 11/03/2020 à 21h

Matisse kra
bts sio

script sql :

```
1 INSERT IGNORE INTO Gare (Nom_Gare) VALUES
2 ('Paris'), ('Istanbul'), ('Lausanne'), ('Milan'),
3 ('Venise'), ('Belgrade'), ('Sofia'), ('Calais');
4
5 INSERT INTO train (Num_train, num_ligne, couleur, tarif)
6 VALUES (242, 2, 'vert', 4000.00);
7
8 INSERT INTO desservir (Num_train, Nom_Gare, Heure_arrivee, Heure_depart) VALUES
9 (242, 'Paris', NULL, '2020-03-10 08:30:00'),
10 (242, 'Lausanne', '2020-03-10 16:15:00', NULL),
11 (242, 'Milan', '2020-03-11 09:45:00', NULL),
12 (242, 'Venise', '2020-03-12 08:00:00', NULL),
13 (242, 'Belgrade', '2020-03-13 19:55:00', NULL),
14 (242, 'Sofia', '2020-03-14 16:00:00', NULL),
15 (242, 'Istanbul', '2020-03-15 16:45:00', NULL);
16
17
18 INSERT INTO train (Num_train, num_ligne, couleur, tarif)
19 VALUES (422, 4, 'violet', 2360.00);
20
21
22 INSERT INTO desservir (Num_train, Nom_Gare, Heure_arrivee, Heure_depart) VALUES
23 (422, 'Calais', NULL, '2020-03-10 09:00:00'),
24 (422, 'Paris', '2020-03-10 12:00:00', NULL),
25 (422, 'Lausanne', '2020-03-11 08:00:00', NULL),
26 (422, 'Milan', '2020-03-11 21:00:00', NULL),
27 (422, 'Venise', '2020-03-12 20:30:00', NULL);
```

			Num_train	Nom_Gare	Heure_arrivee	Heure_depart
☐	 Éditer	 Copier	 Supprimer	242	Belgrade	2020-03-13 19:55:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Istanbul	2020-03-15 16:45:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Lausanne	2020-03-10 16:15:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Milan	2020-03-11 09:45:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Paris	NULL 2020-03-10 08:30:00
☐	 Éditer	 Copier	 Supprimer	242	Sofia	2020-03-14 16:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	242	Venise	2020-03-12 08:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Calais	NULL 2020-03-10 09:00:00
☐	 Éditer	 Copier	 Supprimer	422	Lausanne	2020-03-11 08:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Milan	2020-03-11 21:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Paris	2020-03-10 12:00:00 NULL
☐	 Éditer	 Copier	 Supprimer	422	Venise	2020-03-12 20:30:00 NULL

Conclusion

Ce TP a permis de mettre en pratique la gestion des relations plusieurs-à-plusieurs.
L'utilisation d'une table de jointure (desservir) s'est avérée indispensable pour gérer la

Matisse kra
bts sio

flexibilité des trajets ferroviaires, contrairement à une simple association de un-à-un qui aurait limité le train à seulement deux gares.